



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## **REPORTE DE PRÁCTICA N° 08**

**NOMBRE COMPLETO:** EMILIANO GARCIA OLVERA

**N° de Cuenta:** 314256009

**GRUPO DE LABORATORIO:** 13

**GRUPO DE TEORÍA:** 06

**SEMESTRE** 2026-2

**FECHA DE ENTREGA LÍMITE:** 16/MARZO /2026

**CALIFICACIÓN:** \_\_\_\_\_

**Ejercicio 1: Agregar luz de tipo spotlight para el helicóptero de tal forma que al avanzar (mover con teclado hacia dirección de X negativa ) ilumine con un spotlight en el suelo hacia adelante y al retroceder (mover con teclado hacia dirección de X positiva) ilumine con un spotlight el suelo hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo a la dirección en la que esté moviéndose el helicóptero.**

Para este ejercicio se crean dos variables booleanas que ayudaran a distinguir si el helicoptero esta avanzando o retrocediendo, en cuanto se presiona la tecla UP o DOWN.

En window.cpp

```
theWindow->avanzando = false;
theWindow->retrocediendo = false;
if (key == GLFW_KEY_UP && action == GLFW_PRESS)
{
    theWindow->pos_x -= 2.0;
    theWindow->avanzando = true;
    theWindow->retrocediendo = false;
}
else if (key == GLFW_KEY_DOWN && action == GLFW_PRESS)
{
    theWindow->pos_x += 2.0;
    theWindow->retrocediendo = true;
    theWindow->avanzando = false;
}
```

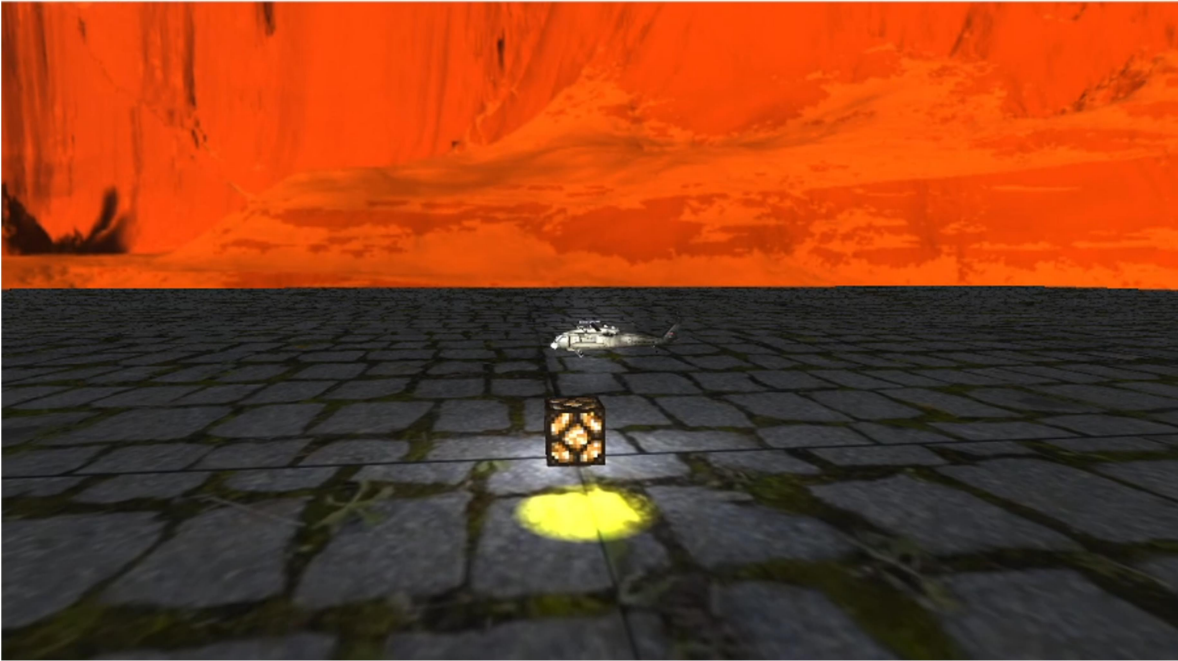
Seguido e esto en main se aplica el cambio en posicion de la linterna.

En main.cpp

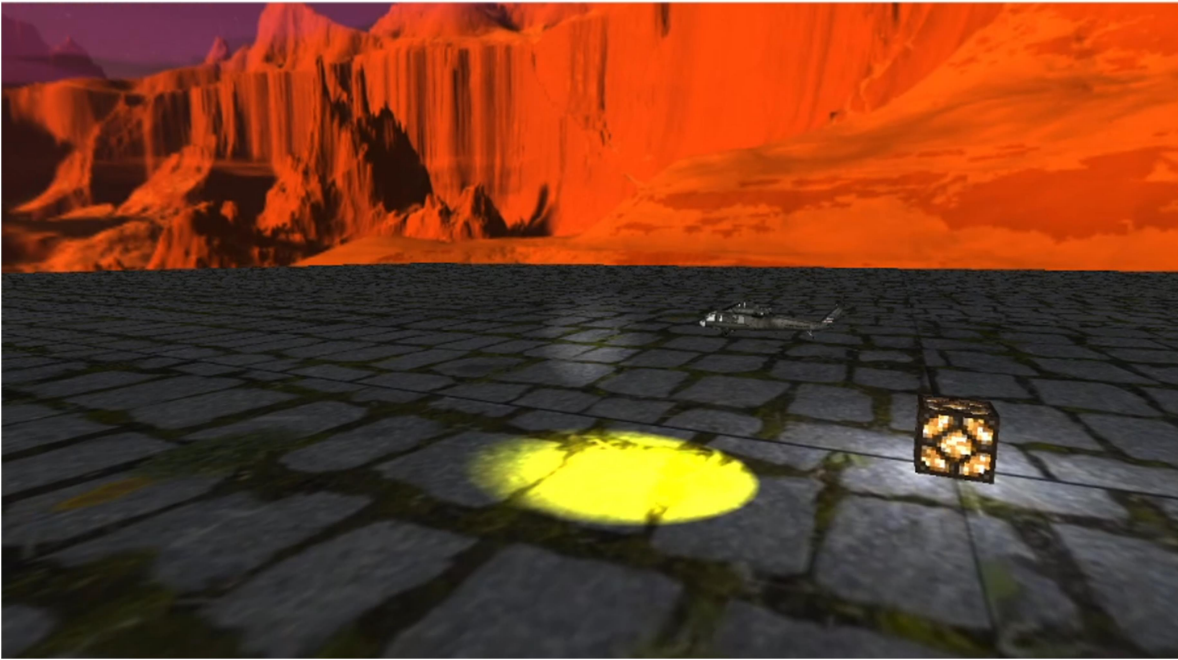
```
if (mainWindow.get_retrocediendo()) {
    spotLights[1].SetFlash(glm::vec3(-0.1f +
mainWindow.get_pos_x(), 4.75f, 6.0), glm::vec3(0.5f, -1.0f, 0.0f));
}
else if (mainWindow.get_avanzando()) {
    spotLights[1].SetFlash(glm::vec3(-0.1f +
mainWindow.get_pos_x(), 4.75f, 6.0), glm::vec3(-0.5f, -1.0f, 0.0f));
}
else spotLights[1].SetFlash(glm::vec3(-0.1f +
mainWindow.get_pos_x(), 4.75f, 6.0), glm::vec3(0.f, -1.0f, 0.0f));
```

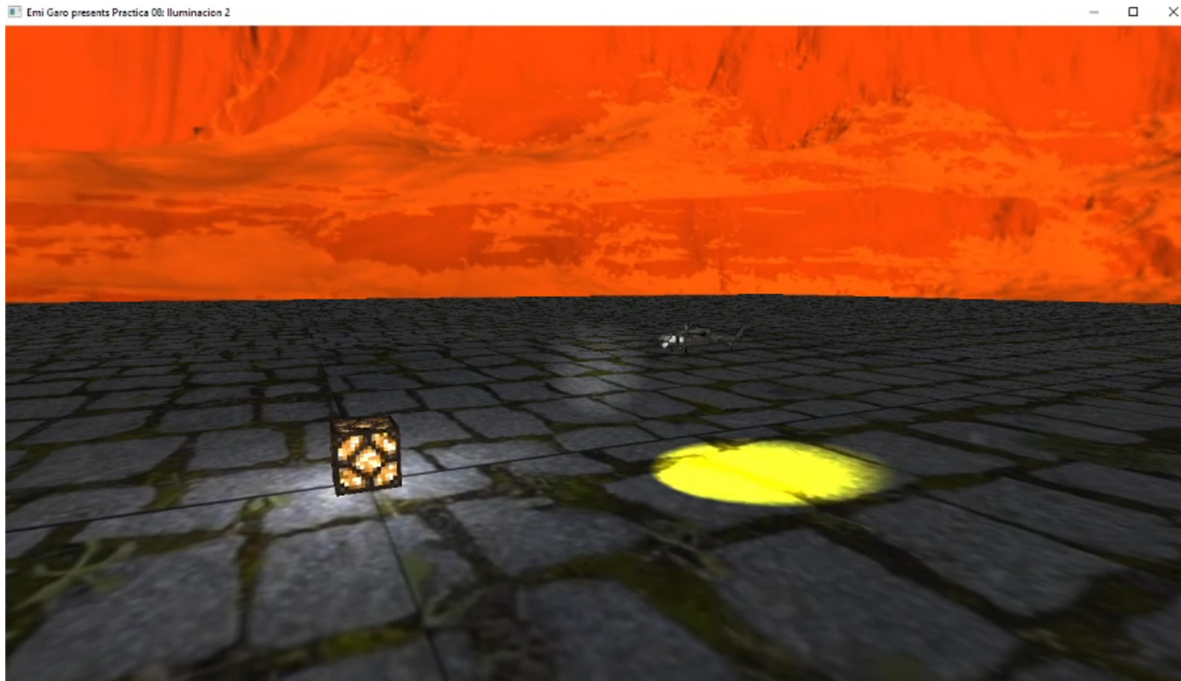
Por lo que el helicoptero ahora cambia su luz al avanzar o retroceder. Y al presionar cualquier otra tecla vuelve a su estado natural.

Emi Gao presents Practica 00: Iluminacion 2



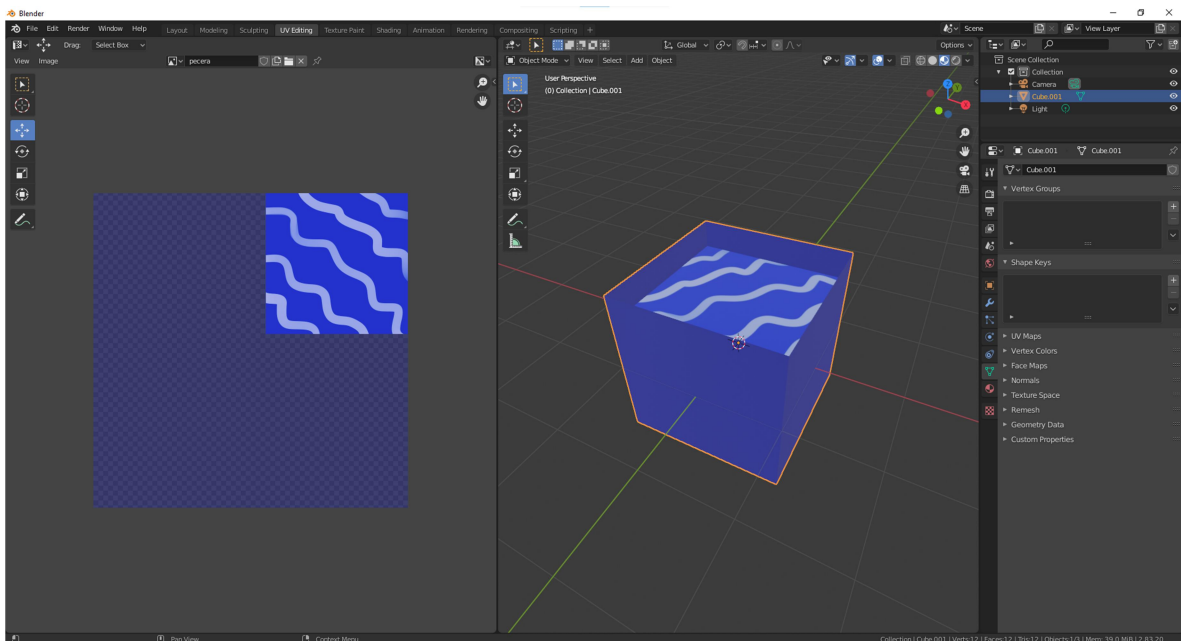
Emi Gao presents Practica 00: Iluminacion 2





**Ejercicio 2: Crear o instanciar una pecera (traslucida) con las normales hacia adentro y en la parte superior imagen de agua e instanciar dentro de la pecera al pez abisal con movimiento de teclado para subir y bajar en diagonal.**

Primero creamos nuestra pecera en blender. Donde la textura sera de color azul con un valor bajo de alfa para simular transparencia en OpenGL. Ademas de esto asignamos una textura de agua sin transparencia para simular la parte superior de la pecera con agua.



Instanciamos los modelos de pecera y pez abisal.

```

// ejercicio 2
// pez cuerpo
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-6.5f, 0.0f, -3.0));
model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
modelaux = model;
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
pez_cuerpo_model.RenderModel();
// pez antena
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 5.0f, 0.0f));
modelaux = model;
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
pez_antena_model.RenderModel();
// pez foco
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 9.0f, 7.0f));
modelaux = model;
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
pez_foco_model.RenderModel();
// pecera
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-7.0f, 2.5f, -3.0));
model = glm::scale(model, glm::vec3(4.0f, 4.0f, 4.0f));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

```

Despues creamos una variable de posicion para el pez la cual obtendra valores al presionar arriba y abajo y agregamos dicha posicion a los ejes 'y' y 'z' del pez. Dandonos el efecto de movimiento. Para esto creamos la variable, su getter y su evento en teclado.

```

GLfloat pez_posicion = 0.0f;

```

```

GLfloat get_pez_posicion() { return pez_posicion; }

```

```

if (key == GLFW_KEY_DOWN)
{
    theWindow->pez_posicion -= 0.1;
}
if (key == GLFW_KEY_UP)
{
    theWindow->pez_posicion += 0.1;
}

```

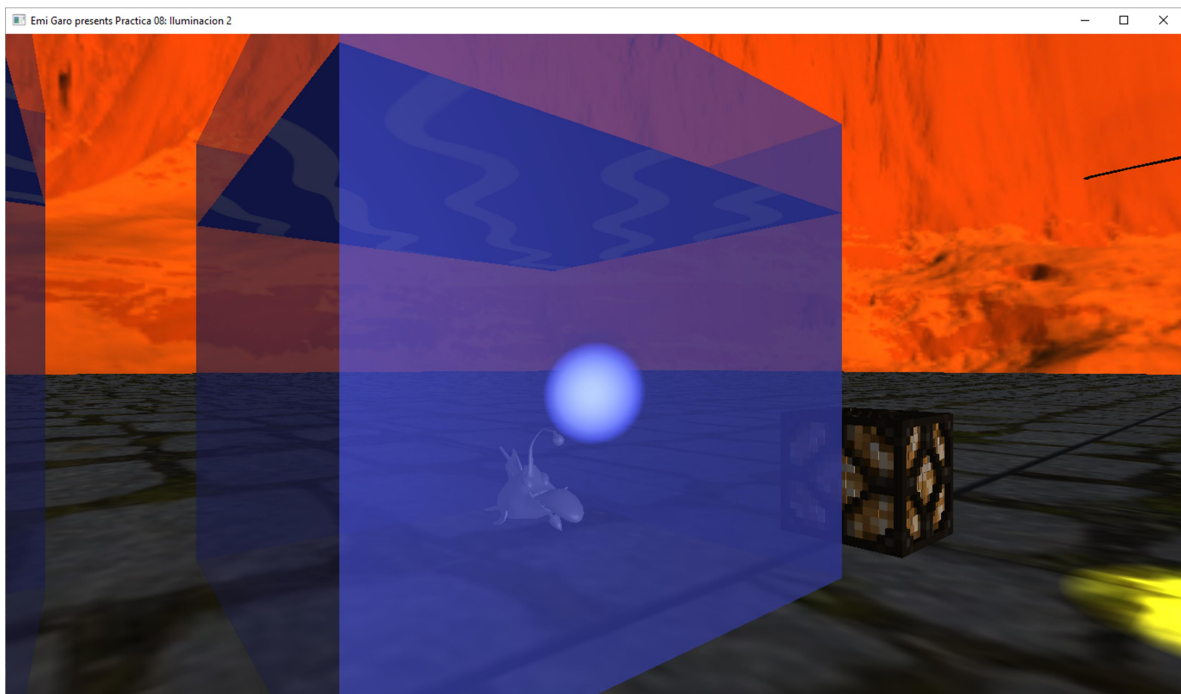


Finalmente aplicamos ficha transformacion al pez y tampoco nos olvidamos de la funciones de transparencia para nuestra pecera.

```
//blending: transparencia o traslucidez
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_FALSE);
glEnable(GL_POLYGON_OFFSET_FILL);
glPolygonOffset(-1.0f, -1.0f);
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);

glEnable(GL_CULL_FACE);
glCullFace(GL_FRONT);
pecera_model.RenderModel();

glCullFace(GL_BACK);
pecera_model.RenderModel();
glDepthMask(GL_TRUE);
glDisable(GL_BLEND);
glDisable(GL_POLYGON_OFFSET_FILL);
glDisable(GL_CULL_FACE);
```



**Ejercicio 3: Agregar una luz de tipo puntual de color azul ligada al bulbo del pez que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara de su práctica 7 como esta luz ( la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual), de tal forma que se pueda ver: las 2 luces apagadas, las 2 luces prendidas, una luz prendida y una luz apagada y viceversa.**

Para este ejercicio extenderemos sobre el ejercicio en clase. Creando una segunda variable del estado de pointlight para el pez. Por lo que realizamos los mismo pasos de crear variable, getter y evento en teclado de la siguiente forma.

```
bool is_pointlight_on = false;
bool is_pointlight2_on = false;

GLfloat get_is_pointlight_on() { return is_pointlight_on; };
GLfloat get_is_pointlight2_on() { return is_pointlight2_on; };

    if (key == GLFW_KEY_Z) {
        theWindow->is_pointlight_on = false;
    }
    if (key == GLFW_KEY_X) {
        theWindow->is_pointlight_on = true;
    }
    if (key == GLFW_KEY_C) {
        theWindow->is_pointlight2_on = false;
    }
    if (key == GLFW_KEY_V) {
        theWindow->is_pointlight2_on = true;
    }
}
```

Nos vamos al main y agregamos las siguientes lineas las cuales modificaran el contador de pointlights depende a los estados para simular el correcto prendido y apagado de luces. Ademas de esto modificaremos el estado del pointlight depende a que luz deberia estar prendida o si ambas lo estan.

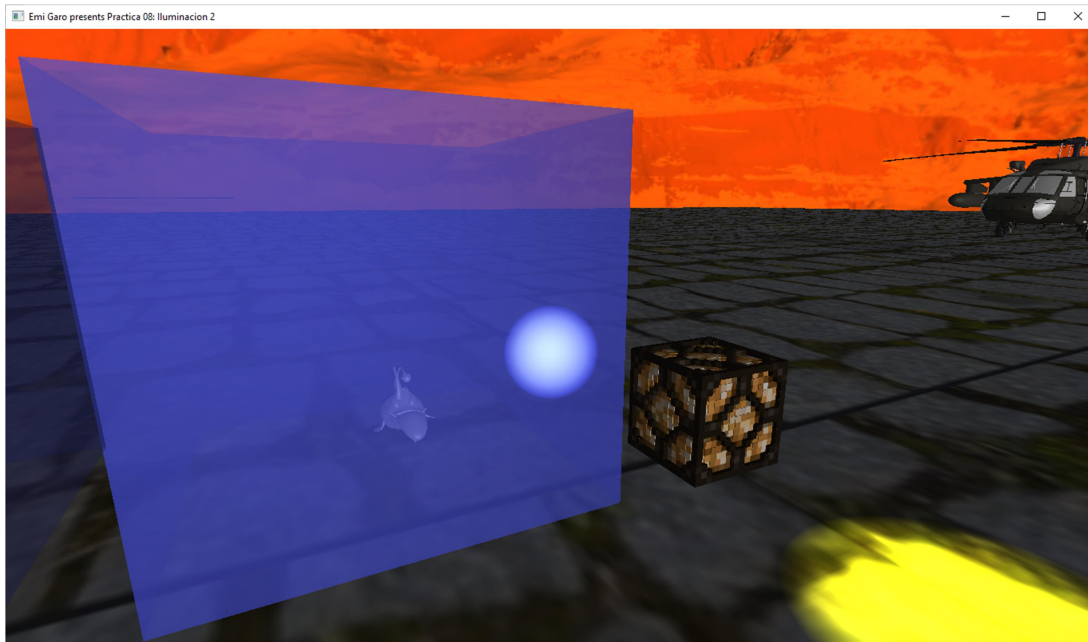
```
// ejercicio 3
// prender apagar lampara
    if (!mainWindow.get_is_pointlight_on() &&
!mainWindow.get_is_pointlight2_on()) {
        pointLightCount = 0;
    }
    else if (mainWindow.get_is_pointlight_on() &&
!mainWindow.get_is_pointlight2_on()) {
        pointLightCount = 1;
        pointLights[0] = PointLight(
            1.0f, 1.0f, 1.0f, // r g b
            0.5f, 1.0f, // a d
            0.0f, 1.0f, 0.0, // x y z
            1.0f, 0.09f, 0.032f); // con lin exp
    }
    else if (!mainWindow.get_is_pointlight_on() &&
mainWindow.get_is_pointlight2_on()) {
        pointLightCount = 1;
        pointLights[0] = PointLight(
            0.0f, 0.0f, 1.0f, // r g b
            0.5f, 1.0f, // a d
            -6.5f, 0.0f, -3.0, // x y z
            1.0f, 0.09f, 0.032f); // con lin exp
    }
    else if (mainWindow.get_is_pointlight_on() &&
mainWindow.get_is_pointlight2_on())
    {
        pointLightCount = 2;
        pointLights[0] = PointLight(
            1.0f, 1.0f, 1.0f, // r g b
```

```

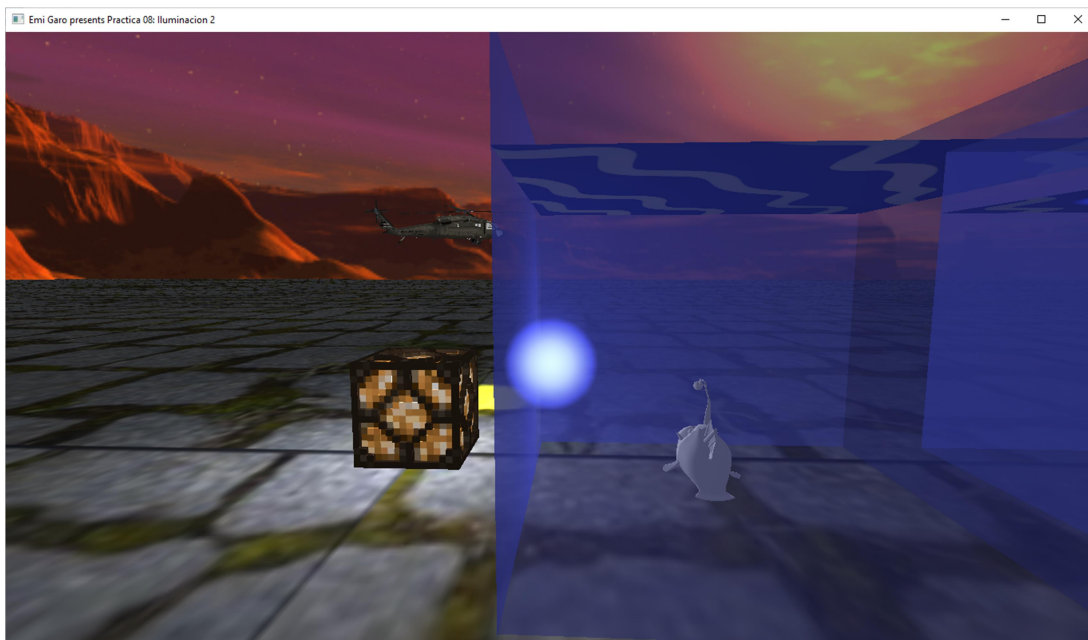
        0.5f, 1.0f,          // a d
        0.0f, 1.0f, 0.0,    // x y z
        1.0f, 0.09f, 0.032f); // con lin exp
pointLights[1] = PointLight(
    0.0f, 0.0f, 1.0f,      // r g b
    0.5f, 1.0f,          // a d
    -6.5f, 0.0f, -3.0,    // x y z
    1.0f, 0.09f, 0.032f); // con lin exp
}

```

Ambas apagadas

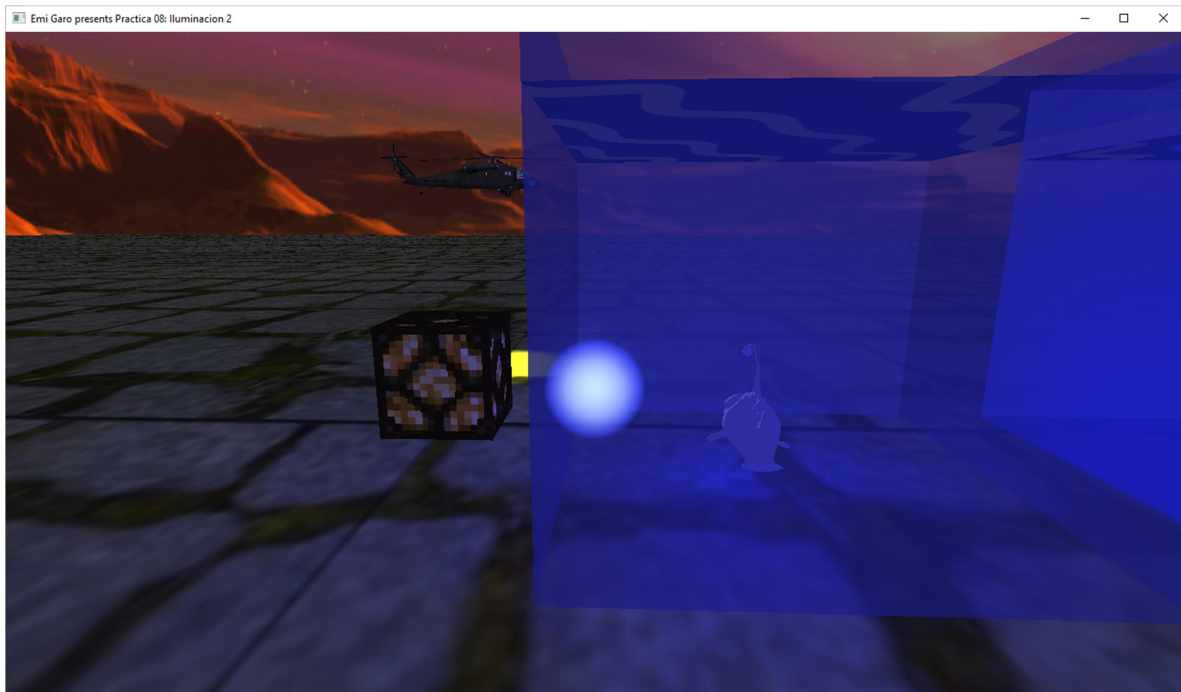


Lampara prendida

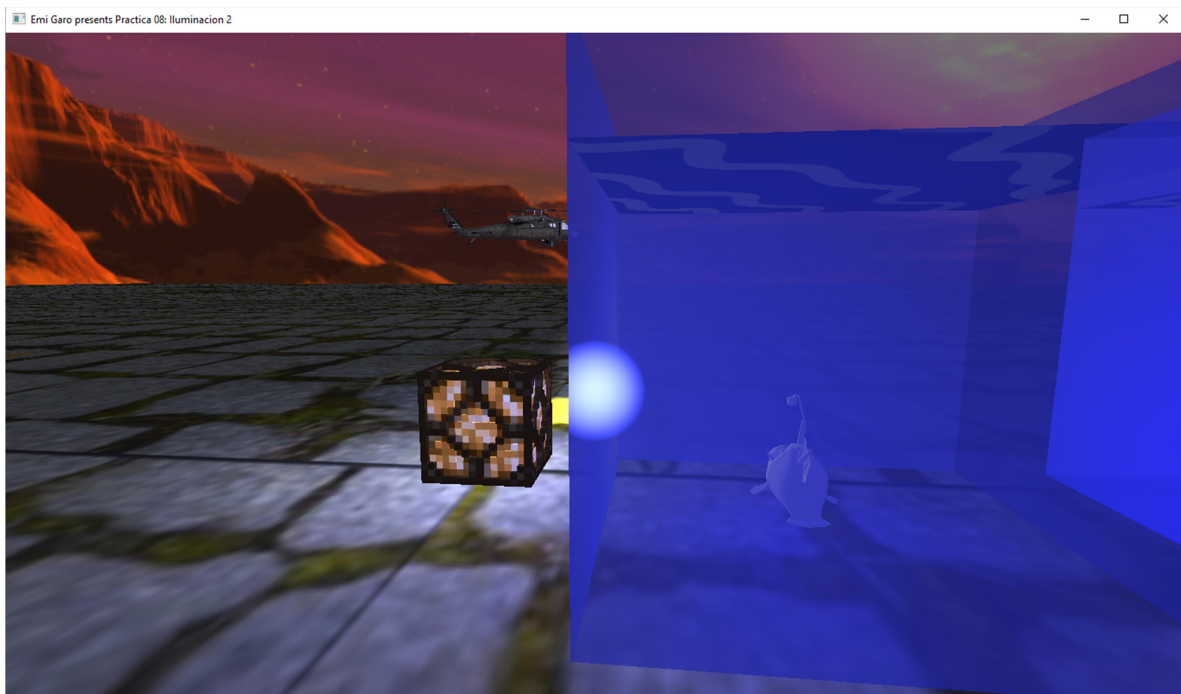




## Bulbo prendido



## Lampara y bulbo prendidos



**Ejercicio 4: Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del bulbo del pez y que ilumine en cualquier direcci3n dentro de la pecera al presionar teclas asignadas al eje x, eje y y eje z.**

Liga de descarga del modelo de pez editado: <https://sketchfab.com/3d-models/pez-abisal-5ca9b50c85424d52a29f47f065f05ab9>

Instanciamos la luz del bulbo como spotlight en la posición del pez y con el color rojo.

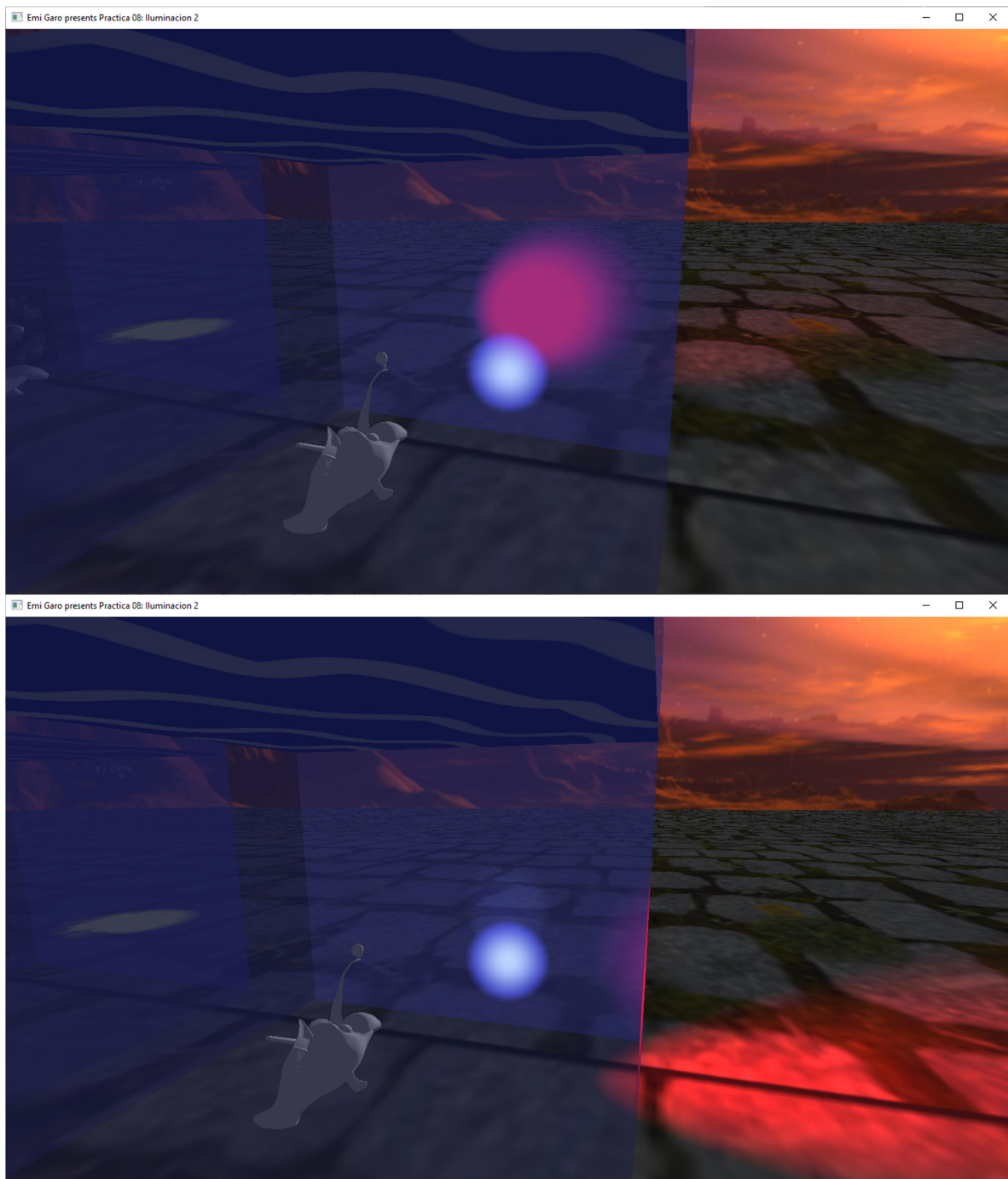
```
// luz bulbo (spotlight)
spotLights[2] = SpotLight(1.0f, 1.0f, 1.0f, // rgb
    6.0f, 3.0f, // ambient diffuse
    -16.5f, 1.2f, -2.5, // pos
    0.0f, 0.0f, 1.0f, // dir
    1.0f, 0.09f, 0.032f, // con lin exp
    25.0f); // edge location
spotLightCount++;
```

Cree otra instancia de pecera y pez para este ejercicio donde lo que nos interesa es el código de renderizado del bulbo.

```
// ejercicio 4
// pez cuerpo
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-16.5f, 0.0f, -3.0));
...
// pez foco
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 9.0f, 7.0f));
glm::mat4 model_rot(1.0);
model_rot = glm::rotate(model, 270 * toRadians, glm::vec3(1.0f,
1.0f, 0.0f));
model_rot = glm::rotate(model_rot, mainWindow.get_articulation_x()
* toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model_rot = glm::rotate(model_rot, mainWindow.get_articulation_y()
* toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model_rot = glm::rotate(model_rot, mainWindow.get_articulation_z()
* toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glm::vec3 bulb_dir = glm::normalize(glm::vec3(model_rot *
glm::vec4(1.0f, 1.0f, 1.0f, 0.0f)));
model = model_rot;
spotLights[2].SetFlash(glm::vec3(-16.5f, 1.3f, -2.2), bulb_dir);
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
pez_foco_model.RenderModel();
modelaux = model;
// pecera
model = glm::mat4(1.0);
```

Como observamos, creamos una variable `model_rot` que obtendrá valores de `get articulation x`, `y`, y `z`, en sus respectivos axis. Con estas la matriz obtendrá las diferentes rotaciones y enseguida asignamos estas a un vector normalizado el cual posteriormente asignaremos al bulbo y ya para terminar, le asignamos las rotaciones también al modelo del bulbo.

Ahora, si movemos las teclas FGHJKL, podemos obtener las diferentes rotaciones del bulbo y spotlight simulando el efecto de iluminación del pez abisal.



## Conclusión

En esta práctica se profundizó en el manejo de iluminación avanzada en OpenGL, integrando múltiples tipos de luces y técnicas de renderizado para lograr efectos visuales más realistas e interactivos.

A través del ejercicio del helicóptero se comprendió cómo una spotlight puede modificar su dirección dinámicamente en función del estado del programa, lo que ilustra la utilidad de ligar propiedades de iluminación al movimiento de objetos en escena.

Con la pecera translúcida se aprendió que el renderizado de objetos transparentes requiere un orden y configuración específicos para obtener una transparencia correcta y sin artefactos.

El ejercicio de las luces puntuales independientes demostró cómo gestionar múltiples fuentes de luz de forma dinámica, activándolas y desactivándolas según el estado del teclado, lo cual es fundamental en aplicaciones interactivas como videojuegos.

Finalmente, la spotlight articulada del pez abisal permitió aplicar transformaciones de rotación sobre la dirección de una luz en tiempo real, reforzando la relación entre las matrices de transformación y el comportamiento de la iluminación en escena.

En conjunto, esta práctica consolidó la comprensión del modelo de iluminación de y su implementación en shaders, así como las buenas prácticas de OpenGL para manejar transparencia, culling y profundidad.

## **Referencias:**

- Apuntes de la clase de teoría y laboratorio de “Computación Gráfica e Interacción Humano Computadora”
  - De Vries, J. (n.d.). Coordinate systems. LearnOpenGL. [learnopengl.com](http://learnopengl.com)
- Conclusión